

Term Project Project

Maria Mauco

1. What I am building

I am building an AI agent to perform natural language queries over complex relational databases using a knowledge graph API. The system extracts database metadata and translates this into a lightweight knowledge graph, allowing the agent to perform multi-table reasoning and querying using external and custom APIs without pulling the entirety of the database or schema into the LLM context window. As a stretch goal (or main goal if professor deems capable), the agent will use graph-editing APIs to dynamically update relationships based on user interactions.

2. Motivations

1. Context window and cost optimization: For every query, feeding a full 200-table schema to an LLM, with or without its data wastes thousands of tokens and increases API costs drastically. If querying repeatedly on the same database, this cost only grows. The knowledge graph acts a smart high level view of the database architecture, allowing for less demanding operations to understand structures and allow for query isolation on only subsets of the original database.

2. Reduces hallucinations: LLMs hallucinate, especially SQL queries, when unclear on what the data/schema looks like or overwhelmed by a large database. By passing a valid concise graph with foreign key relationships, the agent gains an understanding of the data and generates executable SQL.

3. Evaluate any database securely: If the tool only operates on metadata, it allows it to evaluate any database without exposing sensitive information.

4. Any user can ask a question: Allows non-technical users to query database structures using English text

3. Milestones

1. Metadata Extraction and Graph API Construction

Build the database connector to query information_schema. Map tables/columns to nodes and foreign keys to edges. Expose this graph via a queryable API.

Deliverables:

- Database connector script
- Graph API endpoints returning localized subgraphs

2. Agent Integration and Read Action

Build the agent by connecting to an LLM API. Equip the agent with the tools to parse a query, action on the Graph API to extract the relevant subgraph, and formulate the final SQL if querying the database

Deliverable: An agent capable of translating english to SQL using graph-filtered context retrieved from an API

3. Benchmark and Validation

Execute A/B/C testing methodology. Record token usage, response times, and SQL accuracy across varying query complexities

Deliverables: Performance report and comparative metrics between graph assisted vs non. graph assisted agent

4. Stretch Goal (or main): Agent Write Action

Expand the agents tools to allow for graph editing APIs. Allow the agent to add custom semantic edges given updated information. Possibly allow agent to edit the database itself as well if good enough.

Deliverable: Read/Write capable agent with graph and/or database editing capabilities

4. Execution

4.1. Components

1. Database: PostgreSQL[6]

2. Knowledge Graph:

- Option 1: NetworkX (Python)[2]: it's lightweight and nice for localized subgraph extraction

- Option 2: Neo4j [4](leaning more here): It's a dedicated graph database designed for storage and querying. Neo4j has external APIs that can be called from my server

3. API

- Graph Framework Neo4j to query graph

- Hosted REST APIs (in node server) to query database

- Stretch Goal: Neo4j APIs to update graph. the agent will be able to edit the graph directly by adding more custom relationship or edges between tables based on user interaction

- Stretch Goal: Hosted REST APIs to update the database. the agent will be able to perform database operations using endpoints with the information provided by the graph. allowing for faster and more thorough understanding of the data and how to structure without user guidance.

4. LLM Choice: Claude 3.5 Sonnet [1] or GPT-4o [5].

5. Benchmarks and Evaluations

The project's success will be quantified through a A/B/C testing framework measuring cost, efficiency, and accuracy across a standardized dataset.

5.1. Questions

RQ1. Does pre-filtering database context via a knowledge graph API reduce total prompt token consumption compared to passing full-schema or database representations directly to an LLM?

RQ2. How does the graph-assisted context improve multi-table SQL join accuracy and execution success rate compared to baseline non-graph LLM querying?

RQ3. What are the trade-offs in introducing an additional graph API step versus generating SQL from a single full-schema or full database representation prompt?

RQ4. (Stretch Goal) How accurately can an agent identify semantic relationships and utilize write APIs to edit the database structure without introducing invalid schema connections?:

5.2. Tests

Test A (Baseline - No agent or graph): How does an LLM perform given a csv file or raw text of what the database looks like and tasked with answering a complex query or generate a complex SQL query

Test B (Agent): How does an agent perform when given API access to the database and evaluated on the same tasks

Test C (Optimized Agent - with Graph API): How does an agent with actions on the Graph API to retrieve context, which then is passed to the LLM, perform to the same tasks

5.3. Key Performance Indicators (KPIs)

Token consumption, SQL Query execution accuracy, and latency

5.4. Tools

1. LangSmith [3]: integrate natively with LangGraph to trace every agent step. They will automatically record the token usage and latency for Tests A/B/C.

2. Spider Database [7]: to evaluate SQL queries, use this text to SQL dataset to find accuracy benchmarks.

References

- [1] Anthropic. The claude 3 model family: Opus, sonnet, haiku. 2024. [1](#)
- [2] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008. [1](#)
- [3] LangChain AI. Langgraph and langsmith: Building and evaluating agentic workflows, 2024. [2](#)
- [4] Neo4j, Inc. Neo4j graph database, 2024. [1](#)

[5] OpenAI. Hello gpt-4o, 2024. [1](#)

[6] PostgreSQL Global Development Group. *PostgreSQL Documentation*, 2024. [1](#)

[7] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Donglah Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanell Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887*, 2018. [2](#)